

GO Enrichment Analysis Pipeline for Non-Model Organisms

A complete R Markdown pipeline for Gene Ontology (GO) over-representation analysis using `clusterProfiler::enricher()` and `topGO`, designed for any organism in NCBI Gene – no OrgDb package required.

Table of Contents

- 1. [Overview](#)
- 2. [Directory Structure](#)
- 3. [Prerequisites](#)
- 4. [Step 1 – Find Your Organism's TaxID](#)
- 5. [Step 2 – Download NCBI Reference Files](#)
- 6. [Step 3 – Extract Organism-Specific Files](#)
- 7. [Step 4 – Rename Files for the Pipeline](#)
- 8. [Step 5 – Prepare Your DE Gene List](#)
- 9. [Step 6 – Configure the R Markdown Script](#)
- 10. [Step 7 – Install R Packages](#)
- 11. [Step 8 – Run the Pipeline](#)
- 12. [Output Files](#)
- 13. [Adapting for a Different Organism](#)
- 14. [Troubleshooting](#)

Overview

This pipeline performs GO enrichment analysis for any organism with gene annotations in NCBI Gene. It takes a list of differentially expressed (DE) genes (as NCBI Gene IDs or gene symbols) and tests which GO terms are over-represented compared to a background set.

Two complementary methods are run:

- `clusterProfiler::enricher()` – standard Fisher's exact test with BH correction, run separately for Biological Process (BP), Molecular Function (MF), and Cellular Component (CC).
- `topGO` – DAG-aware enrichment using the `weight01` algorithm, which accounts for the hierarchical structure of the Gene Ontology and reduces redundancy between parent and child terms.

Results include 15+ publication-ready visualizations and a head-to-head comparison of both methods.

Directory Structure

After setup, your working directory should look like this:

```
project/
├─ GO_Enrichment_Phypophthalmus.Rmd # The R Markdown pipeline
├─ extract_gene2go.sh                # Bash helper script
├─ extract_gene_info.sh              # Bash helper script
├─ Phypophthalmus_gene2go.tsv        # Organism GO annotations (generated)
├─ Phypophthalmus_gene_info.tsv      # Symbol-to-ID lookup (generated, optional)
├─ de_gene_list.txt                  # Your DE gene list (you provide this)
├─ GO_results/                       # Output directory (created by pipeline)
│   └─ GO_enrichment_BP.tsv
│   └─ GO_enrichment_MF.tsv
│   └─ GO_enrichment_CC.tsv
│   └─ GO_enrichment_combined.tsv
│   └─ topGO_BP.tsv
│   └─ topGO_MF.tsv
│   └─ topGO_CC.tsv
│   └─ enricher_vs_topGO_comparison.tsv
│   └─ plots/
│       └─ dotplot_BP.png
│       └─ barplot_BP.png
│       └─ emap_BP.png
│       └─ ... (all plots as 300-dpi PNGs)
```

Prerequisites

System tools (for bash extraction scripts):

- bash, awk, zcat (standard on macOS and Linux)
- wget or curl (for downloading NCBI files)

R packages (installed in Step 7):

- CRAN: tidyverse, DT, scales, ggrepel, ggwordcloud
- Bioconductor: clusterProfiler, enrichplot, GO.db, AnnotationDbi, topGO

Step 1 – Find Your Organism's TaxID

Every organism in NCBI has a unique Taxonomy ID (TaxID). You need this number to extract the right data from the NCBI reference files.

Option A – Search NCBI Taxonomy:

Go to <https://www.ncbi.nlm.nih.gov/taxonomy/> and search your species name. The TaxID is the number shown next to the organism name.

Option B – Common aquaculture/research TaxIDs:

TaxID	Species	Common name
310915	<i>Pangasianodon hypophthalmus</i>	Striped catfish
7998	<i>Ictalurus punctatus</i>	Channel catfish
8128	<i>Oreochromis niloticus</i>	Nile tilapia
29159	<i>Crassostrea gigas</i>	Pacific oyster
8030	<i>Salmo salar</i>	Atlantic salmon
8049	<i>Oncorhynchus mykiss</i>	Rainbow trout
7955	<i>Danio rerio</i>	Zebrafish
8090	<i>Oryzias latipes</i>	Japanese medaka
31033	<i>Takifugu rubripes</i>	Japanese pufferfish
69293	<i>Gasterosteus aculeatus</i>	Three-spined stickleback
9606	<i>Homo sapiens</i>	Human
10090	<i>Mus musculus</i>	Mouse

Step 2 – Download NCBI Reference Files

Two files are needed from the NCBI Gene FTP server. These are large files that cover ALL organisms – you download them once and extract what you need.

```
# gene2go – maps Gene IDs to GO terms (~1.3 GB compressed)
wget https://ftp.ncbi.nlm.nih.gov/gene/DATA/gene2go.gz

# gene_info – maps Gene IDs to symbols/names (~3.5 GB compressed)
# OPTIONAL: only needed if your DE gene list uses symbols instead of numeric IDs
wget https://ftp.ncbi.nlm.nih.gov/gene/DATA/gene_info.gz
```

Note: On macOS, if wget is not installed, use curl -O instead:

```
curl -O https://ftp.ncbi.nlm.nih.gov/gene/DATA/gene2go.gz
curl -O https://ftp.ncbi.nlm.nih.gov/gene/DATA/gene_info.gz
```

Disk space: You do NOT need to decompress these files. The extraction scripts read `.gz` files directly via `zcat`. If you do decompress them, `gene2go` expands to ~7 GB and `gene_info` to ~18 GB.

Step 3 — Extract Organism-Specific Files

Use the provided bash scripts to filter the NCBI files for your organism. Replace `310915` with your TaxID from Step 1.

```
# Extract GO annotations (REQUIRED)
bash extract_gene2go.sh 310915

# Extract gene info / symbol mapping (OPTIONAL — only if using symbol input)
bash extract_gene_info.sh 310915
```

This produces:

- `gene2go_310915.tsv` — GO annotations for your organism
- `gene_info_310915.tsv` — Symbol/name lookup for your organism

The scripts read the `.gz` files directly (no need to unzip first), and typically finish in under 30 seconds.

Expected output:

```
Filtering TaxID 310915 ...
Done: gene2go_310915.tsv
      203847 lines (including header)
      22541 unique genes
      9823 unique GO terms
```

Tip: If you already decompressed the files, the scripts detect this automatically and read the uncompressed version instead.

Step 4 — Rename Files for the Pipeline

The R Markdown script expects specific filenames. Rename the extracted files to match (or edit the file paths in the Rmd script — see Step 6):

```
mv gene2go_310915.tsv    Phypophthalmus_gene2go.tsv
mv gene_info_310915.tsv  Phypophthalmus_gene_info.tsv    # if generated
```

For a different organism, use a descriptive prefix:

```
# Example for Nile tilapia (TaxID 8128)
mv gene2go_8128.tsv      Oniloticus_gene2go.tsv
mv gene_info_8128.tsv    Oniloticus_gene_info.tsv
```

Step 5 — Prepare Your DE Gene List

Create a plain text file with one gene per line. The pipeline auto-detects whether your input contains numeric NCBI Gene IDs or gene symbols.

Format A — NCBI Gene IDs (recommended, no lookup table needed):

```
# de_gene_list.txt
113547368
113547513
113547290
113546951
113545743
113533484
```

Format B — Gene symbols (requires `gene_info` file from Step 3):

```
# de_gene_list.txt
tlr3
irf9
irf4a
nfkb1
rela
lp1b
```

Rules:

- One gene per line
- Lines starting with # are comments (ignored)
- Inline comments after a tab are also stripped: 113547368 # tlr3
- Blank lines are ignored
- Do not mix IDs and symbols in the same file

Where do these genes come from?

Typically from your DESeq2/edgeR results. For example, in R after running DESeq2:

```
# Extract significant gene IDs from DESeq2 results
sig_genes <- res %>%
  as.data.frame() %>%
  filter(padj < 0.05, abs(log2FoldChange) > 1) %>%
  rownames()

writelines(sig_genes, "de_gene_list.txt")
```

Optional — custom background file:

By default, the pipeline uses all GO-annotated genes as the background (universe). For a more accurate analysis, provide the set of all genes that were *expressed and testable* in your experiment:

```
# All genes that passed DESeq2 independent filtering
all_tested <- rownames(res)[!is.na(res$padj)]
writelines(all_tested, "background_genes.txt")
```

Step 6 — Configure the R Markdown Script

Open `GO_Enrichment_Phypophthalmus.Rmd` and edit these sections:

6a. Organism parameters (line ~89)

```
TAXID      <- 310915                # your TaxID
SPECIES_NAME <- "Pangasianodon hypophthalmus" # display name for plot titles
```

6b. Input file paths (line ~168 and ~275)

```
GENE2GO_FILE <- "Phypophthalmus_gene2go.tsv" # from Step 3/4
GENE_INFO_FILE <- "Phypophthalmus_gene_info.tsv" # from Step 3/4 (optional)
```

6c. DE gene list path (line ~368)

```
DE_GENE_FILE <- "de_gene_list.txt" # your gene list from Step 5
BACKGROUND_FILE <- NULL             # set to "background_genes.txt" if using
                                     # custom background, or leave NULL
```

6d. Enrichment thresholds (line ~95, optional)

```
PVALUE_CUTOFF <- 0.05 # adjusted p-value cutoff
QVALUE_CUTOFF <- 0.2  # q-value cutoff
MIN_GENE_SET  <- 5     # minimum genes per GO term
MAX_GENE_SET  <- 500   # maximum genes per GO term
```

Step 7 — Install R Packages

Run this once in R or RStudio:

```
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")

BiocManager::install(c("clusterProfiler", "enrichplot", "GO.db",
                       "AnnotationDbi", "topGO"))

install.packages(c("tidyverse", "DT", "scales", "ggrepel", "ggwordcloud"))
```

Step 8 — Run the Pipeline

In RStudio:

Open the `.Rmd` file and click **Knit** (or press Ctrl+Shift+K / Cmd+Shift+K).

From the R console:

```
rmarkdown::render("GO_Enrichment_Phypophthalmus.Rmd")
```

From the terminal:

```
Rscript -e 'rmarkdown::render("GO_Enrichment_Phypophthalmus.Rmd")'
```

The pipeline produces a self-contained HTML report and saves all results and plots to the `GO_results/` directory.

Output Files

File	Description
<code>GO_Enrichment_Phypophthalmus.html</code>	Full interactive HTML report
<code>GO_results/GO_enrichment_BP.tsv</code>	<code>enricher()</code> results — Biological Process
<code>GO_results/GO_enrichment_MF.tsv</code>	<code>enricher()</code> results — Molecular Function
<code>GO_results/GO_enrichment_CC.tsv</code>	<code>enricher()</code> results — Cellular Component
<code>GO_results/GO_enrichment_combined.tsv</code>	All ontologies combined with fold enrichment
<code>GO_results/topGO_BP.tsv</code>	topGO results (classic + weight01 + elim) — BP
<code>GO_results/topGO_MF.tsv</code>	topGO results — MF
<code>GO_results/topGO_CC.tsv</code>	topGO results — CC
<code>GO_results/enricher_vs_topGO_comparison.tsv</code>	Method comparison table
<code>GO_results/plots/*.png</code>	All plots as 300-dpi PNGs

Adapting for a Different Organism

The entire pipeline is organism-agnostic. To switch species:

1. **Get TaxID** — look up your organism at <https://www.ncbi.nlm.nih.gov/taxonomy/>
2. **Extract data** — run the bash scripts with the new TaxID:

```
bash extract_gene2go.sh <NEW_TAXID>
bash extract_gene_info.sh <NEW_TAXID>      # optional
```

3. **Rename output files** — or update file paths in the Rmd
4. **Update Rmd parameters** — change TAXID, SPECIES_NAME, and file paths
5. **Provide your DE gene list** — and optionally a custom background
6. **Knit** — everything else runs unchanged

Quick example — switching to Nile tilapia:

```
bash extract_gene2go.sh 8128
mv gene2go_8128.tsv Oniloticus_gene2go.tsv
```

Then in the Rmd:

```
TAXID          <- 8128
SPECIES_NAME   <- "Oreochromis niloticus"
GENE2GO_FILE   <- "Oniloticus_gene2go.tsv"
GENE_INFO_FILE <- "Oniloticus_gene_info.tsv" # if using symbols
```

Troubleshooting

"File not found" errors

Make sure all input files are in the same directory as the .Rmd file, or use absolute paths. Check with:

```
list.files(pattern = "\\\\.tsv$")
```

`AnnotationDbi::select` **masks** `dplyr::select`

The pipeline handles this automatically with `select <- dplyr::select` in the setup chunk. If you still see errors, restart R and knit fresh (cached chunks may hold stale environments).

`enrichplot::barplot` **not found**

The pipeline uses a custom `enrich_barplot()` function that replaces the deprecated `enrichplot::barplot()`. No action needed.

topGO not producing results

- Check that your gene list has enough genes with GO annotations (at least 10–15 is a practical minimum).
- Very small gene lists may not produce significant results with the `weight01` algorithm, which is more conservative than `classic`.

No significant GO terms found

This can happen with small gene lists or sparse annotations. Consider:

- Relaxing thresholds: `PVALUE_CUTOFF <- 0.1`
- Supplementing annotations with eggNOG-mapper or InterProScan
- Using all annotated genes (rather than a custom background) if your background is very restrictive

Slow knitting

The pipeline uses `cache = TRUE` by default, so subsequent knits are fast. The first run may take 2–5 minutes depending on the number of annotated genes. If caching causes stale results after changing inputs, delete the cache:

```
unlink("GO_Enrichment_Phypophthalmus_cache", recursive = TRUE)
```

Citation

If you use this pipeline, please cite the underlying tools:

- **clusterProfiler**: Wu T, et al. (2021). clusterProfiler 4.0: A universal enrichment tool for interpreting omics data. *The Innovation*, 2(3), 100141.
- **topGO**: Alexa A, Rahnenführer J (2009). Gene set enrichment analysis with topGO. *Bioconductor Vignettes*.
- **Gene Ontology**: Gene Ontology Consortium (2023). The Gene Ontology knowledgebase in 2023. *Genetics*, 224(1), iyad031.